

Tutorial sobre testing no automatizado de una aplicación web Spring Boot + Thymeleaf

1. ¿Qué significa probar manualmente una aplicación?

Probar manualmente una aplicación significa **usar la aplicación como lo haría un usuario real** para comprobar que funciona correctamente.

En una aplicación web con Spring Boot y Thymeleaf, esto implica navegar por la interfaz, rellenar formularios, pulsar botones, aplicar filtros, crear registros, modificarlos, borrarlos y comprobar que el resultado coincide con lo esperado.

El objetivo no es “probar al azar”, sino seguir una serie de **casos de prueba planificados y documentados**.

2. Qué partes de la aplicación son más importantes de probar

No todas las partes tienen la misma importancia. Hay que prestar especial atención a las zonas donde el usuario introduce datos, porque es donde más errores suelen aparecer.

2.1. Formularios

Los formularios son una de las partes más importantes que hay que probar.

Por ejemplo:

- Crear un vehículo.
- Editar un conductor.
- Crear una ruta.
- Asignar una ruta a un vehículo o conductor.
- Registrar datos con fechas, números o textos obligatorios.

Hay que comprobar tanto los casos correctos como los incorrectos.

Por ejemplo, en un formulario de vehículo:

- Matrícula válida.
- Matrícula con formato incorrecto.

- Capacidad de carga vacía.
 - Kilometraje negativo.
 - Marca demasiado corta.
 - Campos obligatorios sin rellenar.
-

2.2. Validaciones de datos

También hay que probar que las validaciones funcionan correctamente.

Por ejemplo:

- No permitir una matrícula inválida.
- No permitir una ruta con peso superior a la capacidad del vehículo.
- No permitir fechas vacías si son obligatorias.
- No permitir valores negativos en campos como distancia, peso o kilometraje.

En este punto conviene comprobar dos niveles:

1. **Validación en frontend**, normalmente con HTML5 o JavaScript.
2. **Validación en backend**, usando Spring Boot, `@Valid`, validadores o excepciones.

Aunque el navegador bloquee un dato incorrecto, el backend también debe estar preparado para rechazarlo.

2.3. Campos de filtrado y búsqueda

Los filtros también deben probarse cuidadosamente.

Por ejemplo:

- Buscar rutas entre dos fechas.
- Filtrar vehículos por estado.
- Buscar conductores por nombre.
- Mostrar rutas cuyo coste supere un valor.
- Ordenar o filtrar una tabla con JavaScript.

Hay que probar:

- Filtros con resultados.
 - Filtros sin resultados.
 - Campos vacíos.
 - Fechas incorrectas.
 - Valores límite.
-

2.4. Botones y acciones importantes

También deben probarse las acciones que cambian información en la base de datos:

- Guardar.
- Editar.
- Eliminar.
- Cancelar.
- Confirmar borrado.
- Asignar.
- Finalizar una ruta.
- Cambiar el estado de una entidad.

Especialmente importante: comprobar que el botón de borrar no elimina datos sin confirmación previa.

2.5. Seguridad y roles

Si la aplicación tiene login y roles, también hay que probarlos.

Por ejemplo:

- Un usuario ADMIN puede crear, editar y borrar.
 - Un usuario OPERADOR solo puede consultar y crear.
 - Un usuario no autenticado no puede acceder a páginas protegidas.
 - El menú cambia según el rol.
 - Al cerrar sesión, no se puede volver a páginas privadas usando el botón atrás del navegador.
-

3. Tipos de pruebas que debe hacer el alumnado

Para que el testeo sea completo, se recomienda usar estos tipos de casos:

Caso correcto

Comprueba que la aplicación funciona cuando el usuario introduce datos válidos.

Ejemplo:

Crear un vehículo con una matrícula válida, marca, modelo, capacidad y kilometraje correcto.

Caso incorrecto

Comprueba que la aplicación reacciona bien ante datos no válidos.

Ejemplo:

Intentar crear un vehículo con una matrícula inválida.

Caso límite

Comprueba valores extremos o cercanos al límite permitido.

Ejemplo:

Asignar una ruta con un peso exactamente igual a la capacidad máxima del vehículo.

Caso de seguridad

Comprueba que cada usuario solo puede hacer lo que le corresponde.

Ejemplo:

Intentar acceder como OPERADOR a una página de borrado o edición restringida.

4. Cómo preparar un caso de prueba

Cada caso de prueba debe estar escrito de forma clara para que otra persona pueda repetirlo.

Un buen caso de prueba debe responder a estas preguntas:

- ¿Qué se está probando?
 - ¿En qué ruta o pantalla de la aplicación?
 - ¿Qué datos se van a introducir?
 - ¿Qué pasos hay que seguir?
 - ¿Qué debería ocurrir?
 - ¿Qué ha ocurrido realmente?
-

5. Formato recomendado para documentar las pruebas

El alumnado puede usar una tabla como esta:

ID caso	Funcionalidad	Ruta / pantalla	Datos de entrada	Pasos a seguir	Resultado esperado	Resultado obtenido	Estado
CP-001	Crear vehículo válido	/vehiculos/ nuevo	Matrícula: 1234ABC; Marca: Volvo; Modelo: FH; Capacidad: 12000; Km: 150000	1. Acceder al formulario. 2. Introducir los datos. 3. Pulsar Guardar.	El vehículo se guarda correctamente y aparece en el listado.	El vehículo aparece en el listado con los datos introducidos.	Correcto
CP-002	Crear vehículo con matrícula inválida	/vehiculos/ nuevo	Matrícula: ABC123; Marca: Volvo; Modelo: FH; Capacidad: 12000; Km: 150000	1. Acceder al formulario. 2. Introducir los datos. 3. Pulsar Guardar.	La aplicación muestra un mensaje de error y no guarda el vehículo.	Se muestra error de matrícula inválida.	Correcto

ID caso	Funcionalidad	Ruta / pantalla	Datos de entrada	Pasos a seguir	Resultado esperado	Resultado obtenido	Estado
CP-003	Asignar ruta con exceso de carga	/asignaciones/nueva	Vehículo capacidad: 5000 kg; Ruta peso: 7000 kg	1. Crear o seleccionar una ruta con peso superior. 2. Seleccionar vehículo. 3. Guardar asignación.	La aplicación impide la asignación y muestra un mensaje de error.	La asignación no se guarda y aparece mensaje de capacidad excedida.	Correcto
CP-004	Filtrar rutas entre fechas	/rutas	Desde: 2026-05-01; Hasta: 2026-05-31	1. Acceder al listado de rutas. 2. Introducir fechas. 3. Pulsar Filtrar.	Se muestran solo las rutas dentro del rango indicado.	Se muestran rutas de mayo correctamente.	Correcto
CP-005	Acceso sin login	/vehiculos	Usuario no autenticado	1. Cerrar sesión. 2. Escribir directamente la URL /vehiculos.	La aplicación redirige a la pantalla de login.	Redirige correctamente al login.	Correcto

6. Estados recomendados para cada prueba

En la columna **Estado**, se puede usar una de estas opciones:

Estado	Significado
Correcto	El resultado obtenido coincide con el esperado.
Error detectado	La aplicación no se comporta como debería.
Pendiente	La prueba no se ha podido realizar todavía.
Corregido	El error fue detectado previamente y ya se ha solucionado.

7. Registro de errores detectados

Además de la tabla de casos de prueba, conviene mantener una segunda tabla solo para errores.

ID error	Caso relacionado	Descripción del error	Pasos para reproducirlo	Gravedad	Estado	Solución aplicada
ERR-001	CP-002	La aplicación permite guardar matrículas con formato incorrecto.	Entrar en <code>/vehiculos/nuevo</code> , introducir <code>ABC123</code> y guardar.	Media	Corregido	Se añadió validación con expresión regular en backend y JavaScript.
ERR-002	CP-003	Se permite asignar una ruta con peso superior a la capacidad del vehículo.	Crear ruta de 7000 kg y asignarla a vehículo de 5000 kg.	Alta	Pendiente	Pendiente de revisar la lógica del servicio de asignaciones.

NOTA: También se pueden tener esta lista de errores como *cards* dentro del proyecto en Github, usando el prefijo ERR o FIX. Dentro de la descripción de la card, se pueden incluir todos los datos que se reflejan en esta tabla, aunque no se tenga formato de tabla directamente.

8. Qué evidencias se deben entregar

Para demostrar que has realizado las pruebas, debes entregar:

- Tabla de casos de prueba.
- Tabla de errores detectados (o su reflejo en el proyecto Github)
- Capturas de pantalla de las pruebas más importantes.
- Capturas de mensajes de error.
- Capturas de listados después de crear o modificar datos.
- Breve explicación de los errores encontrados y cómo se corrigieron.

No es necesario capturar la pantalla de absolutamente todo, pero sí de las pruebas más importantes. En la tabla de casos de prueba si deben estar documentados todos.

9. Recomendaciones finales

Antes de entregar el proyecto, cada alumno debería comprobar como mínimo:

1. Que todos los formularios funcionan con datos válidos.
2. Que los formularios rechazan datos incorrectos.
3. Que las reglas de negocio principales se cumplen.
4. Que los filtros y consultas devuelven los datos esperados.
5. Que los botones de editar y borrar funcionan correctamente.
6. Que los roles de usuario tienen los permisos adecuados.
7. Que los mensajes de éxito y error son claros.
8. Que no aparecen páginas de error inesperadas de Spring Boot.

La idea principal es sencilla: **si el usuario puede escribir datos en una pantalla, esa pantalla debe estar probada y documentada.**